

Trabajo Integrador

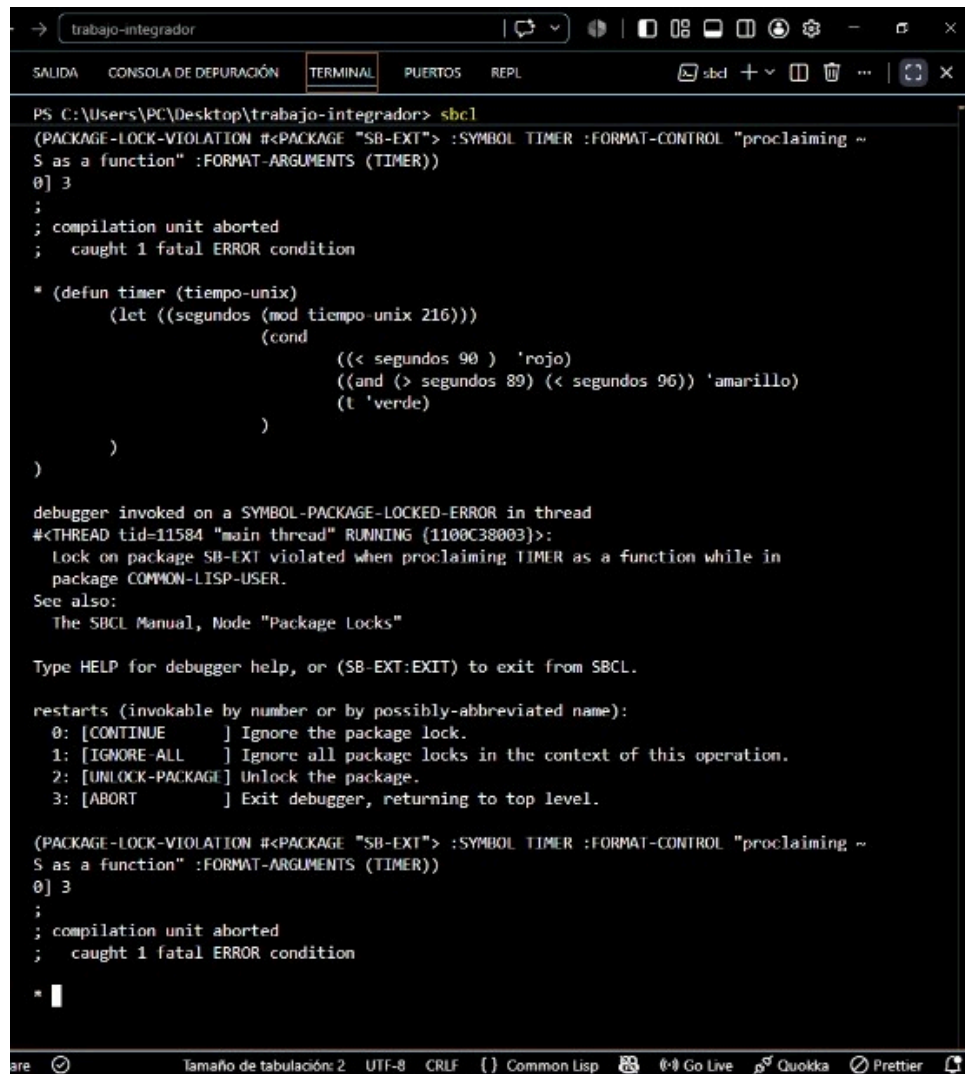
Paradigmas y Lenguajes



INTEGRANTES:

- Prieto Cesar Gabriel
- Vallejos Romero Josefina
- Nuñez, Tobias Nahuel
- Schugurensky, Leandro Daniel
- Sapio, Luciano Agustín

BITÁCORA DE ERRORES



```
PS C:\Users\PC\Desktop\trabajo-integrador> sbcl
(PACKAGE-LOCK-VIOLATION #<PACKAGE "SB-EXT"> :SYMBOL TIMER :FORMAT-CONTROL "proclaiming ~
S as a function" :FORMAT-ARGUMENTS (TIMER))
0] 3
;
; compilation unit aborted
; caught 1 fatal ERROR condition

* (defun timer (tiempo-unix)
  (let ((segundos (mod tiempo-unix 216)))
    (cond
      ((< segundos 90 ) 'rojo)
      ((and (> segundos 89) (< segundos 96)) 'amarillo)
      (t 'verde)
    )
  )
)

debugger invoked on a SYMBOL-PACKAGE-LOCKED-ERROR in thread
#<THREAD tid=11584 "main thread" RUNNING (1100C38003)>:
  Lock on package SB-EXT violated when proclaiming TIMER as a function while in
  package COMMON-LISP-USER.
See also:
  The SBCL Manual, Node "Package Locks"

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
0: [CONTINUE      ] Ignore the package lock.
1: [IGNORE-ALL    ] Ignore all package locks in the context of this operation.
2: [UNLOCK-PACKAGE] Unlock the package.
3: [ABORT         ] Exit debugger, returning to top level.

(PACKAGE-LOCK-VIOLATION #<PACKAGE "SB-EXT"> :SYMBOL TIMER :FORMAT-CONTROL "proclaiming ~
S as a function" :FORMAT-ARGUMENTS (TIMER))
0] 3
;
; compilation unit aborted
; caught 1 fatal ERROR condition

* |
```

Error 1: Conflicto con el nombre de la función **timer**

Error encontrado:

Al compilar el programa se produjo el error **SYMBOL-PACKAGE-LOCKED-ERROR**.

Causa:

La función fue nombrada inicialmente como `timer`. Sin embargo, ese nombre ya se encuentra reservado dentro del paquete interno `SB-EXT`, utilizado por el sistema.

Solución:

Se renombró la función `timer` a `temporizador`, evitando el conflicto con los símbolos reservados del entorno y permitiendo la compilación correcta del programa.

Error 2: Cálculo incorrecto de la distribución por hora

```
(defun distribucion-por-hora()
  (list
    (list 'rojo (*(/ 90 216)100))
    (list 'amarillo (*(/ 6 216)100))
    (list 'verde (*(/ 120 216)100))
  )
)
```

Qué ocurría:

La primera versión de la función `distribucion-por-hora` realizaba divisiones entre números enteros. Como resultado, Lisp devolvía fracciones en lugar de porcentajes decimales, por lo que los valores obtenidos no representan correctamente la distribución de tiempo de cada luz.

Ejemplo del problema:

En lugar de porcentajes expresados con decimales. Al ejecutar la función se obtenían resultados como:

```
: no existe ningún paquete con el nombre QL
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort main loop
Break 2 [3]> (distribucion-por-hora)

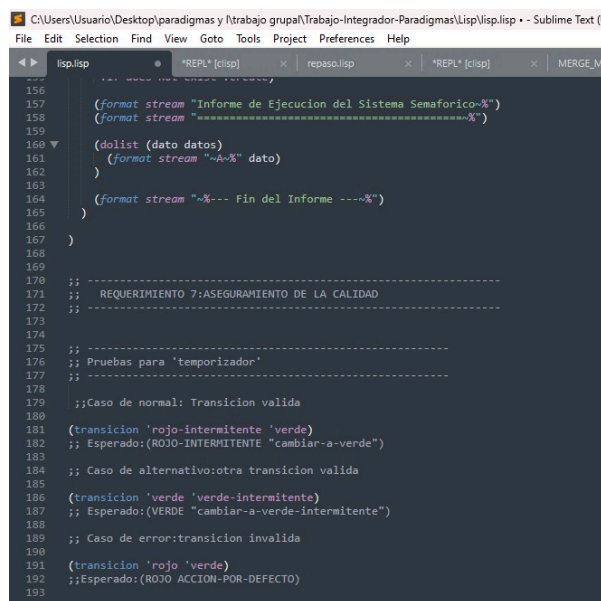
((ROJO 125/3) (AMARILLO 125/3) (VERDE 500/9))
Break 2 [3]> |
```

Solución:

Se modificó la función utilizando `float` en las divisiones para forzar el cálculo en punto flotante. De esta manera los porcentajes se calcularon correctamente y pudieron mostrarse como valores decimales.

```
(defun distribucion-por-hora()
  (list
    (list 'rojo (*(float(/ 90 216))100))
    (list 'amarillo (*(float(/ 6 216))100))
    (list 'verde (*(float(/ 120 216))100))
  )
)
```

Error 3: Reorganización del Requerimiento 7



```
156
157 (format stream "Informe de Ejecucion del Sistema Semaforico-%")
158 (format stream "=====-%")
159
160 (dolist (dato datos)
161   (format stream "~A-%" dato)
162 )
163
164 (format stream "~%--- Fin del Informe -----%")
165 )
166
167
168
169
170 ;; -----
171 ;; REQUERIMIENTO 7:ASEGURAMIENTO DE LA CALIDAD
172 ;; -----
173
174
175 ;; -----
176 ;; Pruebas para 'temporizador'
177 ;; -----
178
179 ;;Caso de normal: Transicion valida
180
181 (transicion 'rojo-intermitente 'verde)
182 ;; Esperado:(ROJO-INTERMITENTE "cambiar-a-verde")
183
184 ;; Caso de alternativo:otra transicion valida
185
186 (transicion 'verde 'verde-intermitente)
187 ;; Esperado:(VERDE "cambiar-a-verde-intermitente")
188
189 ;; Caso de error:transicion invalida
190
191 (transicion 'rojo 'verde)
192 ;;Esperado:(ROJO ACCION-POR-DEFECTO)
193
194
```

Qué ocurría:

Los casos de prueba del Requerimiento 7 fueron agregados inicialmente dentro del archivo principal de Lisp.

Problema encontrado:

La documentación debía entregarse por separado, por lo que la información quedó ubicada en un archivo incorrecto.

Solución:

Se movieron los casos de prueba a la documentación correspondiente y se realizaron nuevos commits para conservar los cambios correctamente en el repositorio.

Error 4: Uso de `dolist` en la generación del informe

Error encontrado:

La primera versión de la función `generar-informe` utilizaba `dolist` para recorrer la lista de eventos.

Causa:

Durante la implementación se eligió `dolist` por ser una forma sencilla de recorrer listas. Sin embargo, la consigna prohibía el uso de estructuras iterativas como `loop`, `dolist`, `dotimes` y `while`.

Solución:

Se reemplazó la iteración por una función recursiva auxiliar que procesa los elementos de la lista uno por uno, respetando las restricciones del paradigma funcional exigidas por el trabajo práctico.

Fase 3: Haskell

1. Common Lisp tiene un tipado dinámico. Haskell tiene un tipado estático estricto.

¿Cómo modelaron los colores del semáforo?

Expliquen qué es un Algebraic Data Type (ADT) y cómo el compilador de Haskell impide pasar un estado inválido antes de que el programa se ejecute.

Algebraic Data Type (ADT)

Un Algebraic Data Type (ADT) es una forma de definir un tipo cuyos valores posibles están limitados a un conjunto de alternativas previamente declaradas. Esto permite modelar conceptos del dominio de manera segura, ya que una variable de ese tipo solo puede tomar alguno de los valores especificados.

Al conocer todos los valores válidos de un tipo, el compilador puede detectar errores antes de ejecutar el programa. Si se intenta utilizar un valor que no pertenece al tipo definido, la compilación falla.

¿Cómo impide Haskell que se pasen estados inválidos?

Haskell posee un sistema de tipos estático y estricto. Como el compilador conoce todos los valores válidos de cada tipo durante la compilación, puede verificar que las funciones reciban únicamente datos correctos.

Si se intenta pasar un valor que no pertenece al tipo esperado, el programa no compilará. De esta manera, muchos errores se detectan antes de la ejecución, aumentando la seguridad y la confiabilidad del código.

¿Cómo modelamos los colores del semáforo?

Modelamos los colores del semáforo utilizando ADTs. Definimos tres tipos de datos:

```
data EstadoActual = EnRojo | RojoIntermitente | EnVerde | VerdeIntermitente | EnAmarillo  
| AmarilloIntermitente
```

```
data ColorDestino = DestRojo | DestRojoIntermitente | DestVerde | DestVerdeIntermitente  
| DestAmarillo | DestAmarilloIntermitente
```

```
data ResultadoTransicion = ResultadoTransicion EstadoActual String
```

EstadoActual representa el color en el que se encuentra actualmente el semáforo, incluyendo tanto los estados estables como los intermedios (intermitentes). ColorDestino representa el color al que se desea realizar la transición, con el prefijo Dest en cada constructor para evitar conflictos de nombres con EstadoActual dentro del mismo módulo. ResultadoTransicion agrupa los dos valores devueltos por la función transicion: el estado actual del semáforo y la acción que debe realizarse.

Se decidió utilizar tipos distintos para EstadoActual y ColorDestino porque representan conceptos diferentes dentro del dominio del problema. En la versión de Common Lisp esta diferencia se reflejaba mediante símbolos distintos (en-rojo, en-verde, etc. para el estado actual, y rojo, verde, etc. para el destino). En Haskell, esta distinción se expresa de manera explícita mediante tipos separados, lo que aporta mayor claridad y seguridad al programa. El prefijo Dest en ColorDestino es además una consecuencia práctica del sistema de tipos de Haskell, que no permite que dos tipos del mismo módulo compartan el nombre de un constructor.

2. Expliquen cómo afecta la Evaluación Perezosa (Lazy Evaluation) de Haskell al procesamiento de los tiempos del temporizador si tuvieran un flujo infinito de eventos de tráfico.

¿Cómo afecta la Evaluación Perezosa (Lazy Evaluation) de Haskell al procesamiento de los tiempos del temporizador si existiera un flujo infinito de eventos de tráfico?

La Evaluación Perezosa (Lazy Evaluation) significa que Haskell no calcula una expresión hasta que su resultado es realmente necesario.

Gracias a esta característica, Haskell puede procesar un flujo infinito de eventos de tráfico sin consumir toda la memoria disponible. En lugar de evaluar todos los timestamps de una vez, los procesa de manera incremental, calculando únicamente los valores que se necesitan en cada momento.

Esto permite aplicar la función temporizador sobre una secuencia infinita de tiempos sin inconvenientes, ya que los datos se generan y consumen a medida que son requeridos. En un lenguaje con evaluación estricta, como Common Lisp, sería necesario implementar mecanismos adicionales para evitar cargar toda la secuencia en memoria y provocar un consumo excesivo de recursos.

REQUERIMIENTO 7: ASEGURAMIENTO DE LA CALIDAD

TRANSICION			
CASOS			ESPERADO
CASO NORMAL	Transición válida	(transicion 'rojo-intermitente 'verde)	(ROJO-INTERMITENTE "cambiar-a-verde")
CASO ALTERNATIVO	Otra transición válida	(transicion 'rojo 'verde)	(ROJO ACCION-POR-DEFECTO)
CASO DE ERROR	Entrada invalida	(transicion 'agus 'azul)	(AGUS ACCION-POR-DEFECTO)

```
Break 3 [4]> (transicion 'rojo-intermitente 'verde)
(ROJO-INTERMITENTE "cambiar-a-verde")
Break 3 [4]> (transicion 'rojo 'verde)
(ROJO ACCION-POR-DEFECTO)
Break 3 [4]> (transicion 'agus 'azul)
(AGUS ACCION-POR-DEFECTO)
```

TEMPORIZADOR			
CASOS			ESPERADO
CASO NORMAL	Tiempo dentro de un rango rojo	(temporizador 50)	ROJO
CASO ALTERNATIVO	Tiempo negativo (sigue siendo válido)	(temporizador -4)	AMARILLO
CASO DE ERROR	Parámetro invalido	(temporizador 'jojo)	JOJO is not a real number

```
Break 3 [4]> (temporizador 50)
ROJO
Break 3 [4]> (temporizador -4)
AMARILLO
Break 3 [4]> (temporizador 'jojo)
*** - MOD: JOJO is not a real number
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead.
```

DURACION-CICLO			
CASOS			ESPERADO
CASO NORMAL	Suma de los 6 tiempos	(duracion-ciclo 90 3 120 3 6 3)	225
CASO ALTERNATIVO	Otra combinación valida	(duracion-ciclo 90 3 120 3 0 3)	219
CASO DE ERROR	Más de 6 parámetros	(duracion-ciclo 90 3 120 3 6 3 10)	se han entregado demasiados argumentos a DURACION-CICLO

```
Break 6 [7]> (duracion-ciclo 90 3 120 3 6 3)

225
Break 6 [7]> (duracion-ciclo 90 3 120 3 0 3)

219
Break 6 [7]> (duracion-ciclo 90 3 120 3 6 3 10)

*** - EVAL/APPLY: se han entregado demasiados argumentos a DURACION-CICLO
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
ABORT      :R5      Abort debug loop
ABORT      :R6      Abort debug loop
ABORT      :R7      Abort main loop
```

RECOMENDACIÓN-CICLO			
CASOS			ESPERADO
CASO NORMAL	Ciclo largo	(recomendacion-ciclo (duracion-ciclo 90 3 120 3 6 3))	"rango del ciclo larga, optimo: 35-150 segundos"
CASO ALTERNATIVO	Ciclo óptimo	(recomendacion-ciclo 35)	"duración óptima"
CASO DE ERROR	Parámetro invalido	(recomendacion-ciclo 'j')	j is not a real number

```
Break 8 [9]> (recomendacion-ciclo (duracion-ciclo 90 3 120 3 6 3))

"rango del ciclo larga, optimo: 35-150 segundos"
Break 8 [9]> (recomendacion-ciclo 35)

"duracion optima"
Break 8 [9]> (recomendacion-ciclo 'j')

*** - <: j is not a real number
Es posible continuar en los siguientes puntos:
USE-VALUE  :R1      Input a value to be used instead.
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
ABORT      :R5      Abort debug loop
ABORT      :R6      Abort debug loop
ABORT      :R7      Abort debug loop
ABORT      :R8      Abort debug loop
ABORT      :R9      Abort debug loop
ABORT      :R10     Abort main loop
```

CICLOS-POR-TIEMPO			
CASOS			ESPERADO
CASO NORMAL	Daría como resultado:16 ciclos y 0 segundos restantes	(ciclos-por-tiempo 3600 90 3 120 3 6 3)	16 ; 0
CASO ALTERNATIVO	Otra forma:con 7 ciclos y 20 segundos	(ciclos-por-tiempo 1000 50 10 60 0 20 0)	7 ; 20
CASO DE ERROR	No se puede dividir por 0	(ciclos-por-tiempo 3600 0 0 0 0 0 0)	division by zero

```
Break 12 [13]> (ciclos-por-tiempo 3600 90 3 120 3 6 3)
16 ;
0
Break 12 [13]> (ciclos-por-tiempo 1000 50 10 60 0 20 0)
7 ;
20
Break 12 [13]> (ciclos-por-tiempo 3600 0 0 0 0 0 0)

*** - FLOOR: division by zero
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
ABORT      :R5      Abort debug loop
ABORT      :R6      Abort debug loop
ABORT      :R7      Abort debug loop
ABORT      :R8      Abort debug loop
ABORT      :R9      Abort debug loop
ABORT      :R10     Abort debug loop
ABORT      :R11     Abort debug loop
ABORT      :R12     Abort debug loop
ABORT      :R13     Abort main loop
```

DISTRIBUCION –POR-HORA			
CASOS			ESPERADO
CASO NORMAL	Ejecución correcta	(distribucion-por-hora 90 120 6)	((ROJO 41.666664) (VERDE 55.555557) (AMARILLO 2.777778))
CASO DE ERROR	División por 0,no es posible	(distribucion-por-hora 0 0 0)	division by zero

```
Break 13 [14]> (distribucion-por-hora 90 120 6)

((ROJO 41.666664) (VERDE 55.555557) (AMARILLO 2.777778))
Break 13 [14]> (distribucion-por-hora 0 0 0)

*** - /: division by zero
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
ABORT      :R5      Abort debug loop
ABORT      :R6      Abort debug loop
ABORT      :R7      Abort debug loop
ABORT      :R8      Abort debug loop
ABORT      :R9      Abort debug loop
ABORT      :R10     Abort debug loop
ABORT      :R11     Abort debug loop
ABORT      :R12     Abort debug loop
ABORT      :R13     Abort debug loop
ABORT      :R14     Abort main loop
```

REGISTRAR-CAMBIO			
CASOS			ESPERADO
CASO NORMAL	Caso válido que muestra por pantalla el cambio de estado del semáforo junto con la fecha y hora	(registrar-cambio 1749499200 'rojo 'verde)	Tiempo 1749499200: la luz ha cambiado de ROJO a VERDE
CASO ALTERNATIVO	Otro caso válido	(registrar-cambio 1749499320 'verde 'amarillo)	Tiempo 1749499320: la luz ha cambiado de VERDE a AMARILLO
CASO DE ERROR	El parámetro no corresponde al tiempo Unix	(registrar-cambio 'hola 'rojo 'verde)	ERROR: el tiempo debe ser un timestamp Unix numerico

```
* (registrar-cambio 1749499200 'rojo 'verde)
Tiempo 1749499200: la luz ha cambiado de ROJO a VERDE
T
* (registrar-cambio 1749499320 'verde 'amarillo)
Tiempo 1749499320: la luz ha cambiado de VERDE a AMARILLO
T
* (registrar-cambio 'hola 'rojo 'verde)
ERROR: el tiempo debe ser un timestamp Unix numerico.
NIL
*
```

Bibliografía

- [Haskell Documentación](#)
- Bibliografía aportada por la cátedra
- [Lisp Localtime doc](#)
- [Quicklisp beta](#)